

Документирование API с помощью OpenAPI (Swagger): топ 10 продвинутых лайфхаков

В этом гайде мы рассмотрим 10 продвинутых лайфхаков, которые помогут вам максимально эффективно использовать OpenAPI для документирования ваших API и повысить уровень профессионализма в работе с этим инструментом!



[@OPENSTUDYIT](#)

1. Повторное использование компонентов (Schemas, Parameters)

OpenAPI позволяет повторно использовать определенные компоненты, такие как схемы данных (schemas) и параметры запросов, что значительно упрощает поддержку и обновление спецификаций API.

Почему это важно?

- Избегаете повторного описания одинаковых элементов.
- Изменения в одном месте автоматически применяются во всех местах использования.
- Спецификация становится более чистой и понятной.

А: Определите общие компоненты в разделе components вашей спецификации.

```
components:
  schemas:
    User:
      type: object
      properties:
        id:
          type: integer
        name:
          type: string
```

Б: Ссылайтесь на них с помощью \$ref в местах использования.

```
paths:
  /users/{id}:
    get:
      summary: Получить информацию о пользователе
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Успешный ответ
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/User'
```

Советы

- Структурируйте компоненты по категориям для лучшей организации.
- Используйте понятные и однозначные названия для компонентов.

2. Продвинутое моделирование данных (oneOf, anyOf, allOf)

Эти конструкции позволяют описывать сложные структуры данных, которые могут принимать различные формы.

- oneOf: объект должен соответствовать одной из перечисленных схем.
- anyOf: объект должен соответствовать по крайней мере одной из схем.
- allOf: объект должен соответствовать всем перечисленным схемам.

Почему это важно?

- Позволяет конкретно и ёмко описать различные модели данных похожих объектов. Клиенты и серверы могут точно проверить соответствие данных.

Пример использования oneOf. В API объект Транспорт, в поле Тип Транспорта, клиент может отправить либо Машина либо Лодка. От этого в объекте Транспорт появятся другие дополнительные поля (либо колёса, либо пропеллер).

```
components:
  schemas:
    Vehicle:
      type: object
      discriminator:
        propertyName: vehicleType
      required:
        - vehicleType
      properties:
        vehicleType:
          type: string
      oneOf:
        - $ref: '#/components/schemas/Car'
        - $ref: '#/components/schemas/Boat'
    Car:
      type: object
      properties:
        vehicleType:
          type: string
          enum: [Car]
        wheels:
          type: integer
    Boat:
      type: object
      properties:
        vehicleType:
          type: string
          enum: [Boat]
        propeller:
          type: boolean
```

3. Документирование аутентификации и авторизации

OpenAPI предоставляет возможности для описания схем безопасности, таких как API ключи, OAuth2, JWT и другие.

Почему это важно?

- Потребители API будут точно знать, как проходить аутентификацию.
- Инструменты могут автоматически настроить аутентификацию при генерации «клиентов».

А:Определите схемы безопасности:

```
components:
```

```
  securitySchemes:
```

```
    BearerAuth:
```

```
      type: http
```

```
      scheme: bearer
```

```
      bearerFormat: JWT
```

Б:Примените их к API или отдельным операциям:

```
security:
```

```
  - BearerAuth: []
```

Советы

- Всегда обновляйте информацию о безопасности при изменении методов аутентификации.
- Предоставляйте примеры токенов или ключей (в учебных целях) для облегчения понимания.

4. Использование расширений (x-extensions)

Расширения (поля, начинающиеся с x-) позволяют добавлять пользовательскую информацию в спецификацию OpenAPI (подсказки, примеры кода, дополнительные заголовки и т.д.).

Почему это важно?

- Добавление дополнительной информации, специфичной для вашей организации или инструментов.
- Передача данных, необходимых для других процессов или инструментов.

Пример добавления расширения:

paths:

/users:

get:

summary: Получить список пользователей

x-rateLimit: 1000

Советы

- Соблюдайте единый стиль именования для ваших расширений.
- Документируйте назначение ваших расширений для команд и пользователей.

5. Версионирование API

Практика управления изменениями в API путем назначения версий.

Почему это важно?

- Позволяет поддерживать разные версии API для разных клиентов.
- Облегчает управление обновлениями и устаревшими функциями.

Способы версионирования:

1. В URL: `/v1/users`
2. В заголовках: добавление заголовка `API-Version`
3. В параметрах запросов: `GET /users?version=1`

Пример с использованием URL:

paths:

`/v1/users:`

get:

summary: Получить список пользователей (версия 1)

Советы

- Определите стратегию версионирования с самого начала.
- Уведомляйте пользователей об устаревании версий заранее.

6. Продвинутое аннотации и комментарии

Добавление подробных описаний, примеров и заметок в спецификацию для улучшения понимания API.

Почему это важно?

- Делает спецификацию более понятной для разработчиков.
- Подробные описания помогают избежать недопонимания (для сложных объектов и ответов).

Добавление описаний и примеров:

```
paths:
```

```
  /users:
```

```
    get:
```

```
      summary: Получить список пользователей
```

```
      description: Возвращает список всех зарегистрированных  
пользователей.
```

```
      responses:
```

```
        '200':
```

```
          description: Успешный ответ
```

```
          content:
```

```
            application/json:
```

```
              schema:
```

```
                type: array
```

```
                items:
```

```
                  $ref: '#/components/schemas/User'
```

```
      examples:
```

```
        example1:
```

```
          summary: Пример списка пользователей
```

```
          value:
```

```
            - id: 1
```

```
              name: "Иван"
```

```
            - id: 2
```

```
              name: "Мария"
```

7. Объединение нескольких спецификаций API

Разделение большой спецификации на несколько файлов и их объединение при необходимости.

Почему это важно?

- Легче работать с меньшими файлами.
- Разные команды могут работать над своими частями API.

Используйте `$ref` для ссылок на внешние файлы:

```
paths:
```

```
  /users:
```

```
    $ref: './paths/users.yaml'
```

```
components:
```

```
  schemas:
```

```
    $ref: './components/schemas.yaml'
```

Советы

- Структурируйте файлы по функциональным областям или следуя вашей логике, чтобы не запутаться, когда объем файлов будет большим.

8. Использование Callbacks для асинхронных API

Позволяют описывать асинхронные операции, при которых сервер инициирует обратный вызов к клиенту.

Почему это важно?

- Клиенты могут правильно реализовывать обработку обратных вызовов. Все взаимодействия будут задокументированы

Пример описания callback:

paths:

/subscribe:

post:

requestBody:

required: true

content:

application/json:

schema:

type: object

properties:

callbackUrl:

type: string

responses:

'201':

description: Подписка успешно создана

callbacks:

onEvent:

'{\$request.body#/callbackUrl}':

post:

requestBody:

.....

responses:

'200':

description: Успешное получение события по подписке

Советы

- Используйте переменные, такие как {\$request.body#/callbackUrl}, для ссылок на данные из запроса.
- Четко опишите, какие данные и когда будут отправляться в обратном вызове(requestBody).

9. Валидация данных с помощью Schemas в OpenAPI

Schemas в OpenAPI основаны на стандарте JSON Schema и позволяют определить, какие данные ожидаются, их типы, форматы и ограничения. Это обеспечивает автоматическую валидацию данных.

Почему это важно?

- Гарантирует, что клиент и сервер обмениваются данными в ожидаемом формате. Предотвращает передачу некорректных данных.

Самые частые элементы валидации:

1. `enum`: указывает список допустимых значений для поля.
2. `format`: определяет предопределенный формат данных для поля.
3. `pattern`: задает регулярное выражение для проверки значения поля.
4. `default`: устанавливает значение по умолчанию для поля, если оно не указано.

Пример:

```
components:
```

```
  schemas:
```

```
    UserProfile:
```

```
      type: object
```

```
      required:
```

```
        - email
```

```
      properties:
```

```
        email:
```

```
          type: string
```

```
          format: email
```

```
        status:
```

```
          type: string
```

```
          enum:
```

```
            - active
```

```
            - inactive
```

```
          default: 'active'
```

```
        phoneNumber:
```

```
          type: string
```

```
          pattern: '^\+\d{1,3}-\d{3}-\d{4}$'
```

```
          example: '+1-555-1234'
```

9. Валидация данных с помощью Schemas в OpenAPI

Schemas в OpenAPI основаны на стандарте JSON Schema и позволяют определить, какие данные ожидаются, их типы, форматы и ограничения. Это обеспечивает автоматическую валидацию данных.

Почему это важно?

- Гарантирует, что клиент и сервер обмениваются данными в ожидаемом формате. Предотвращает передачу некорректных данных.

Самые частые элементы валидации:

1. `enum`: указывает список допустимых значений для поля.
2. `format`: определяет предопределенный формат данных для поля.
3. `pattern`: задает регулярное выражение для проверки значения поля.
4. `default`: устанавливает значение по умолчанию для поля, если оно не указано.

Пример:

```
components:
```

```
  schemas:
```

```
    UserProfile:
```

```
      type: object
```

```
      required:
```

```
        - email
```

```
      properties:
```

```
        email:
```

```
          type: string
```

```
          format: email
```

```
        status:
```

```
          type: string
```

```
          enum:
```

```
            - active
```

```
            - inactive
```

```
          default: 'active'
```

```
        phoneNumber:
```

```
          type: string
```

```
          pattern: '^\+\d{1,3}-\d{3}-\d{4}$'
```

```
          example: '+1-555-1234'
```

10. Обеспечение согласованности и качества с помощью линтинга и стайлгайдов

Линтинг — это автоматическая проверка вашей OpenAPI спецификации на ошибки, несоответствия и отклонения от стандартов. Стайлгайды — это набор правил и рекомендаций по оформлению и структуре спецификации, которые помогают поддерживать единый стиль.

Хотя Swagger Editor предоставляет инструменты для базовой валидации OpenAPI, использование дополнительных инструментов линтинга и стайлгайдов позволяет достичь более высокого уровня качества и единого стандарта.

Почему это важно?

- Спецификации, оформленные в одном стиле, легче читать и понимать. Меньше времени тратится на поиск проблем и согласование стиля в команде.
 - Автоматическая проверка помогает находить и исправлять ошибки.
1. Интегрируйте линтер (например, [Spectral](#)) в ваш рабочий процесс (договоритесь с командой разработки).
 2. Определите правила, которым должна соответствовать ваша спецификация (например, обязательное наличие описаний у всех операций, единый стиль именования и т.д.). Опишите их в спецификации инструмента Spectral.
 3. Проверяйте спецификации с помощью Spectral, на основе ваших правил.

Пример стайлгайда:

```
extends: spectral:oas
```

```
rules:
```

```
  operation-description:
```

```
    description: "Каждая операция должна иметь описание"
```

```
    given: "$.paths[*][*]"
```

```
    then:
```

```
      field: description
```

```
      function: truthy
```

Советы:

- Организуйте интеграцию с системами контроля версий и CI/CD. Настройте линтинг спецификации при каждом обновлении проекта.
- Обучайте свою команду и поддерживайте стайлгайд.